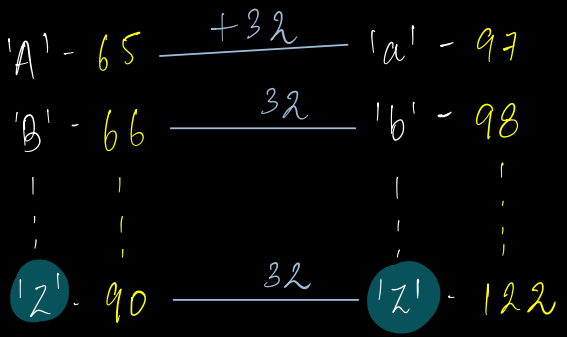
Strings Introduction

1) Intro
2) Flip
3) Count array[ ]
4) Reverse substring
5) Longest Palindromic Substring
6) Prefix Strings.

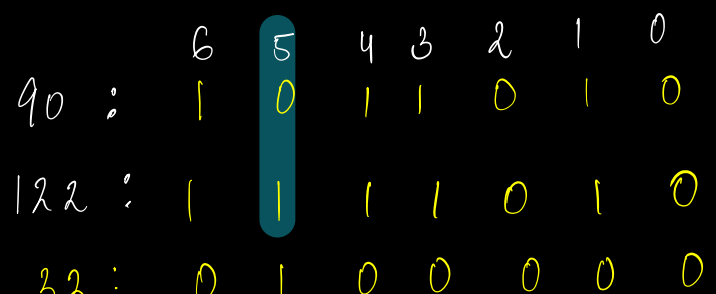
Strings :- Sequence of characters / Array of characters.



'0' - 48
'1' - 49
'9' - 57

Set - unset
1 — 0

unset - set
0 - 1



$2^5 = 32$

Between upper case & lower case characters, 5th bit is different

XOR with 32 will toggle 5th bit

Given a string S, every character is small or capital

We have to toggle.

Ex S = a B c A F d — AbCafD

(0) (1) (2) (3) (4) (5)

① No if else
② No ternary
③ No functions
④ No extra spa

```
string toggle(string S){
    int N= S.size();
    for(int i=0; i<N; i++){
        if(S[i]>=65 && S[i]<=90){
            //S[i] is upper case
            S[i]+=32
        }
        else{
            //S[i] is small case
            S[i]-=32
        }
    }
    return S
}
```

1 - 0
0 - 1

S[i]^=32

Not changing

S[i] - 5th I/O

32 - 0 0 0 1 0 0 0 0 0

S[i]^32 → changing 5th bit.

Q2) Given a string, which contains any lowercase characters. sort given string.

26 distinct characters.

Ex S = d a b a c d b → a a b b c d d

Sol 1:- 1) Use sorting function. TC - O(N log N)

Sol 2:- int cut[26] = {0}

// freq of 26 characters.

cut[0] - freq of 'a' : 97

cut[1] - freq of 'b' : 98

cut[2] - freq of 'c' : 99

cut[25] - freq of 'z' : 122

// S[i] will be stored at S[i]-97 index

S = d a b a e d b

	i — i + 'a'
cut[0] : 2	a a b b d d e
cut[1] : 2	//
cut[2] : 0	final ans
cut[3] : 2	
cut[4] : 1	

Pseudo Code :- (Independent of input)

```
1) int cut[26] = {0}
   int N = S.size();
   for(int i=0; i<N; i++){
      int ind = S[i] - 97;
      cut[ind]++;
   }                                  } O(N)
   String ans = "";
   for(int i=0; i<26; i++){           } N
      // ith index - i + 'a'
      char ch = i + 'a'
      for(int j=1; j<=cut[i]; j++){
         ans += ch
      }
   }
   return ans
```

TC - O(N), SC - O(1)

{ Count Sort, SC - O(1) → O(N) } Advanced batch.

```
      i i i i i i i
Str = d a b a e d b
```

cut[26]: store freq of every char

0	1	2	3	4	.....	25
2	2	0	2	1	0	0	0 ...
'a'	'b'	'c'	'd'	'e'	.....	
i	i	i	i	i	i	i ....

ch = 4 + 'a' = e

Final ans = a a b b d d e

```
1) int cut[26] = {0}
   int N = S.size();
   for(int i=0; i<N; i++){
       int ind = S[i] - 97;
       cut[ind] ++;
   }
   String ans = "";
   for(int i=0; i<26; i++){
       //ith index - i + 'a'
       char ch = i + 'a'
       for(int j=1; j<=cut[i]; j++;
           ans += ch
       }
   }
   return ans
```

```
for(int i=0; i<26; i++){
    //ith index - i + 'a'
    char ch = i + 'a'
    for(int j=1; j<=cut[i]; j++){
        ans += ch
    }
}
```

i	j	# of iterations
0	[1, cut[0]]	cut[0] +
1	[1, cut[1]]	cut[1] +
2	. .	cut[2] +
:	:	: +
25	...	cut[25]

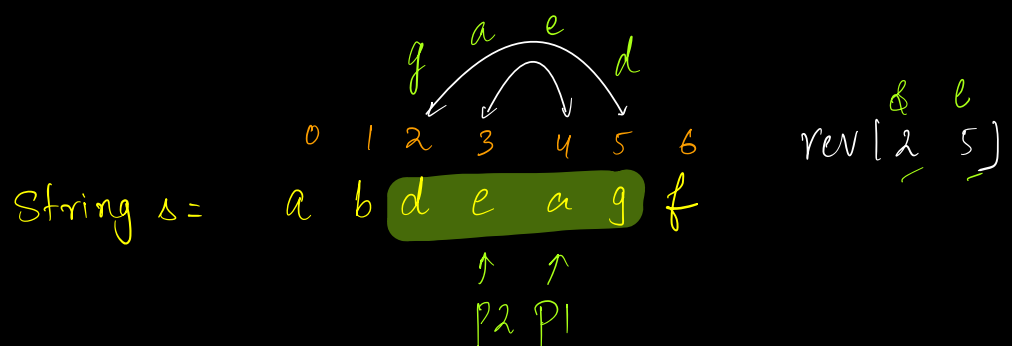
total # iterations = cut[0] + cut[1] + cut[2] + ... cut[25]

⇒ (N)

Q3) Given a string. Reverse a part of string.

Substrings :- Concept of is same as Subarrays.

1) Continuous part of string
2) Full string can be as substring
3) A single character is also a substring.




```
                                    p1      p2-1
String reverse(char s[], int s, int e){
    int p1 = s, p2 = e
    while(p1 <= p2){
        Swap(s[p1], s[p2])
        p1++
        p2--
    }
}
```

→ String

Q4) Given a char[]. Reverse word by word. Note: B/w every word, there is a space.

// No extra space
// No inbuit.
// No extra spaces at start & end.

Ex1 str :- love hate data structures.

Str :- structures data hate love

Ex1 Srikar is a Scaler student

Str: student Scaler a is Srikar

Ex1 str :- love hate data structures.

Reverse entire String

↳ sertcurts atad etah evol

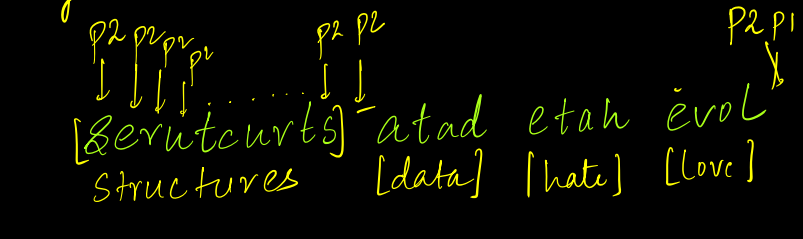
before space, after space

p1 p2 p1 p2 p1 p2 p1 p2

Reverse every word

Step 1:- Reverse entire string

Step 2:- p1 = 0, p2 = 0




```
while (p2 < N) {
    while(p2 < N && av[p2] != ' ') {
        p2++
    }
    // p2 will stop at space
    // We need to reverse [p1 to p2-1]
    reverse(av, p1, p2-1)
    p1 = p2+1
    p2 = p1
}
```

Q5) Prefix Strings.

For a given string, all substrings starting at index 0.

Ex Deepthi (0 1 2 3 4 5 6)

D, De, Dee, Deep, Deept, Deepth, Deepthi

Q6) Given 2 strings S1 & S2, Calculate Longest common prefix string.

Ex1 S1: abacadabra
S2: abacus
} abac, len = 4

Ex2 S1: Satyagrah
S2: satyam
} Satya, len = 5

Ex3 S1: salman
S2: driver
} ans = 0

selman
selmo
} len = 4

```
int i = 0, len = 0
while(i < s1.size() && i < s2.size()){
    if(s1[i] == s2[i]){
        i++
        len++
    }
    else{
        break
    }
}
return len
```

TC: O(N), SC: O(1)

Q7) Given a string check, if a given substring is palindrome or not. (L → R) = (R → L)

Ex s1 = a c d c a } Palindrome

Ex s1 = m a d a y a d a m } Palindrome

P1 P1P2 P2

TC: O(N)

```
bool isPalindrome(string s, p1, p2){
    while(p1 < p2){
        if(s[p1] == s[p2]){
            p1++
            p2--
        }
        else{
            return false
        }
    }
    return true
}
```

Q8) Given a string, Calculate Length of longest Palindromic substring.

Ex1

0	1	2	3	4	5
a	b	a	c	a	b

len = 3

↳ len = 5

Ex2:

0	1	2	3	4
a	b	c	d	e

↳ Len = 1

Solution :- For every substring, check Palindrome or not & find max length.

```
ans = 0
for(int s = 0; s < N; s++){
    for(int e = s; e < N; e++){
        // s[s...e] is a substring
        if(isPalindrome(str, s, e)){
            int len = e - s + 1
            if(len > ans){
                ans = len
            }
        }
    }
}
```

Selmon

[2, 2]

[2, 3)

⋮

[s...e]

TC - $O(N^3)$, SC - $O(1)$

Next approach - will discuss in next session ☺